

Question 1

```
In [38]: import pandas as pd

# To Load the dataset
df = pd.read_excel(r"F:\Semester 4\Machine Learning\CIA 1\QP_Data_road_accident_dat

# Showing first 10 rows
print(df.head(10))

# To separate numerical and categorical variables
num_vars = df.select_dtypes(include=['int64', 'float64']).columns
cat_vars = df.select_dtypes(include=['object']).columns

print("\nNumerical Variables:", list(num_vars))
print("Categorical Variables:", list(cat_vars))

# To check data types of all columns
print("\nData Types:\n", df.dtypes)
```

	Country	Year	Month	Day of Week	Time of Day	Urban/Rural	Road Type	\
0	USA	2002	October	Tuesday	Evening	Rural	Street	
1	UK	2014	December	Saturday	Evening	Urban	Street	
2	USA	2012	July	Sunday	Afternoon	Urban	Highway	
3	UK	2017	May	Saturday	Evening	Urban	Main Road	
4	Canada	2002	July	Tuesday	Afternoon	Rural	Highway	
5	India	2010	May	Monday	Evening	Urban	Street	
6	China	2010	March	Monday	Afternoon	Rural	Street	
7	USA	2016	July	Friday	Afternoon	Rural	Main Road	
8	Japan	2014	August	Thursday	Afternoon	Rural	Highway	
9	USA	2007	April	Monday	Evening	Urban	Highway	

	Weather Conditions	Visibility Level	Number of Vehicles Involved	...	\
0	Windy	NaN	1	...	
1	Windy	168.311358	3	...	
2	Snowy	341.286506	4	...	
3	Clear	489.384536	2	...	
4	Rainy	348.344850	1	...	
5	Snowy	479.216834	2	...	
6	Foggy	386.176217	3	...	
7	Foggy	75.608688	3	...	
8	Rainy	387.828675	3	...	
9	Foggy	443.965408	3	...	

	Number of Fatalities	Emergency Response Time	Traffic Volume	\
0	2	58.625720	7412.752760	
1	1	58.041380	4458.628820	
2	4	NaN	9856.915064	
3	3	48.554014	4958.646267	
4	4	18.318250	3843.191463	
5	4	8.205994	360.951795	
6	3	17.851663	7607.804705	
7	2	46.740367	6061.407002	
8	2	NaN	3793.850542	
9	3	NaN	1140.429308	

	Road Condition	Accident Cause	Insurance Claims	Medical Cost	\
0	Wet	Weather	4	40499.856980	
1	Snow-covered	Mechanical Failure	3	6486.600073	
2	Wet	Speeding	4	29164.412980	
3	Icy	Distracted Driving	3	25797.212570	
4	Icy	Distracted Driving	8	15605.293920	
5	Dry	Speeding	7	NaN	
6	Wet	Weather	9	NaN	
7	Dry	Speeding	8	4262.755621	
8	Snow-covered	Mechanical Failure	5	37624.775980	
9	Snow-covered	Distracted Driving	0	15801.190080	

	Economic Loss	Region	Population Density
0	22072.878500	Europe	3866.273014
1	9534.399441	North America	2333.916224
2	58009.145120	South America	4408.889129
3	20907.151300	Australia	2810.822423
4	13584.060760	South America	3883.645634
5	45995.605250	South America	3626.074027
6	52342.431810	Asia	3408.182341

7	70652.223520	South America	408.296453
8	13724.630950	Europe	2058.898279
9	61948.862750	Australia	1840.206143

[10 rows x 30 columns]

Numerical Variables: ['Year', 'Visibility Level', 'Number of Vehicles Involved', 'Speed Limit', 'Driver Alcohol Level', 'Driver Fatigue', 'Pedestrians Involved', 'Cyclists Involved', 'Number of Injuries', 'Number of Fatalities', 'Emergency Response Time', 'Traffic Volume', 'Insurance Claims', 'Medical Cost', 'Economic Loss', 'Population Density']

Categorical Variables: ['Country', 'Month', 'Day of Week', 'Time of Day', 'Urban/Rural', 'Road Type', 'Weather Conditions', 'Driver Age Group', 'Driver Gender', 'Vehicle Condition', 'Accident Severity', 'Road Condition', 'Accident Cause', 'Region']

Data Types:

Country	str
Year	int64
Month	str
Day of Week	str
Time of Day	str
Urban/Rural	str
Road Type	str
Weather Conditions	str
Visibility Level	float64
Number of Vehicles Involved	int64
Speed Limit	int64
Driver Age Group	str
Driver Gender	str
Driver Alcohol Level	float64
Driver Fatigue	int64
Vehicle Condition	str
Pedestrians Involved	int64
Cyclists Involved	int64
Accident Severity	str
Number of Injuries	int64
Number of Fatalities	int64
Emergency Response Time	float64
Traffic Volume	float64
Road Condition	str
Accident Cause	str
Insurance Claims	int64
Medical Cost	float64
Economic Loss	float64
Region	str
Population Density	float64
dtype:	object

C:\Users\dell\AppData\Local\Temp\ipykernel_56908\227936333.py:11: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
cat_vars = df.select_dtypes(include=['object']).columns
```

Question 2a

In [40]: `df.describe()`

Out[40]:

	Year	Visibility Level	Number of Vehicles Involved	Speed Limit	Driver Alcohol Level	Driver I
count	132000.000000	125400.000000	132000.000000	132000.000000	104893.000000	132000.
mean	2011.973348	275.104755	2.501227	74.544068	0.110463	0.
std	7.198624	129.946180	1.117272	26.001448	0.066818	0.
min	2000.000000	50.001928	1.000000	30.000000	0.000002	0.
25%	2006.000000	162.422604	2.000000	52.000000	0.053893	0.
50%	2012.000000	274.808977	3.000000	74.000000	0.107667	1.
75%	2018.000000	388.070736	3.000000	97.000000	0.161436	1.
max	2024.000000	499.999646	4.000000	119.000000	0.249989	1.

```
In [ ]: # To identify variable with highest variation (we are using standard deviation)
variation = df[num_vars].std().sort_values(ascending=False)
print("Highest variation:", variation.index[0])
```

Highest variation: Economic Loss

Question 2b

```
In [ ]: mean_medical = df['Medical Cost'].mean()
median_medical = df['Medical Cost'].median()

print("Mean:", mean_medical)
print("Median:", median_medical)

if mean_medical > median_medical:
    print("Right-skewed distribution (positive skew)")
elif mean_medical < median_medical:
    print("Left-skewed distribution (negative skew)")
else:
    print("Symmetric distribution")
```

Mean: 22999.326534801297

Median: 22687.63412

Right-skewed distribution (positive skew)

Question 3a

```
In [10]: # To count missing values and percentage for each column
missing = df.isnull().sum()
missing_percent = (missing / len(df)) * 100
```

```
pd.DataFrame({'Missing': missing, 'Percent': missing_percent})
```

Out[10]:

	Missing	Percent
Country	0	0.000000
Year	0	0.000000
Month	0	0.000000
Day of Week	0	0.000000
Time of Day	0	0.000000
Urban/Rural	0	0.000000
Road Type	0	0.000000
Weather Conditions	0	0.000000
Visibility Level	6600	5.000000
Number of Vehicles Involved	0	0.000000
Speed Limit	0	0.000000
Driver Age Group	0	0.000000
Driver Gender	0	0.000000
Driver Alcohol Level	27107	20.535606
Driver Fatigue	0	0.000000
Vehicle Condition	0	0.000000
Pedestrians Involved	0	0.000000
Cyclists Involved	0	0.000000
Accident Severity	0	0.000000
Number of Injuries	0	0.000000
Number of Fatalities	0	0.000000
Emergency Response Time	13176	9.981818
Traffic Volume	6600	5.000000
Road Condition	0	0.000000
Accident Cause	0	0.000000
Insurance Claims	0	0.000000
Medical Cost	25021	18.955303
Economic Loss	13200	10.000000
Region	0	0.000000
Population Density	6600	5.000000

Question 3b

```
In [55]: df['Visibility Level'] = df['Visibility Level'].fillna(df['Visibility Level'].median())
df['Traffic Volume'] = df['Traffic Volume'].fillna(df['Traffic Volume'].median())
df['Population Density'] = df['Population Density'].fillna(df['Population Density'].median())
```

Question 3c

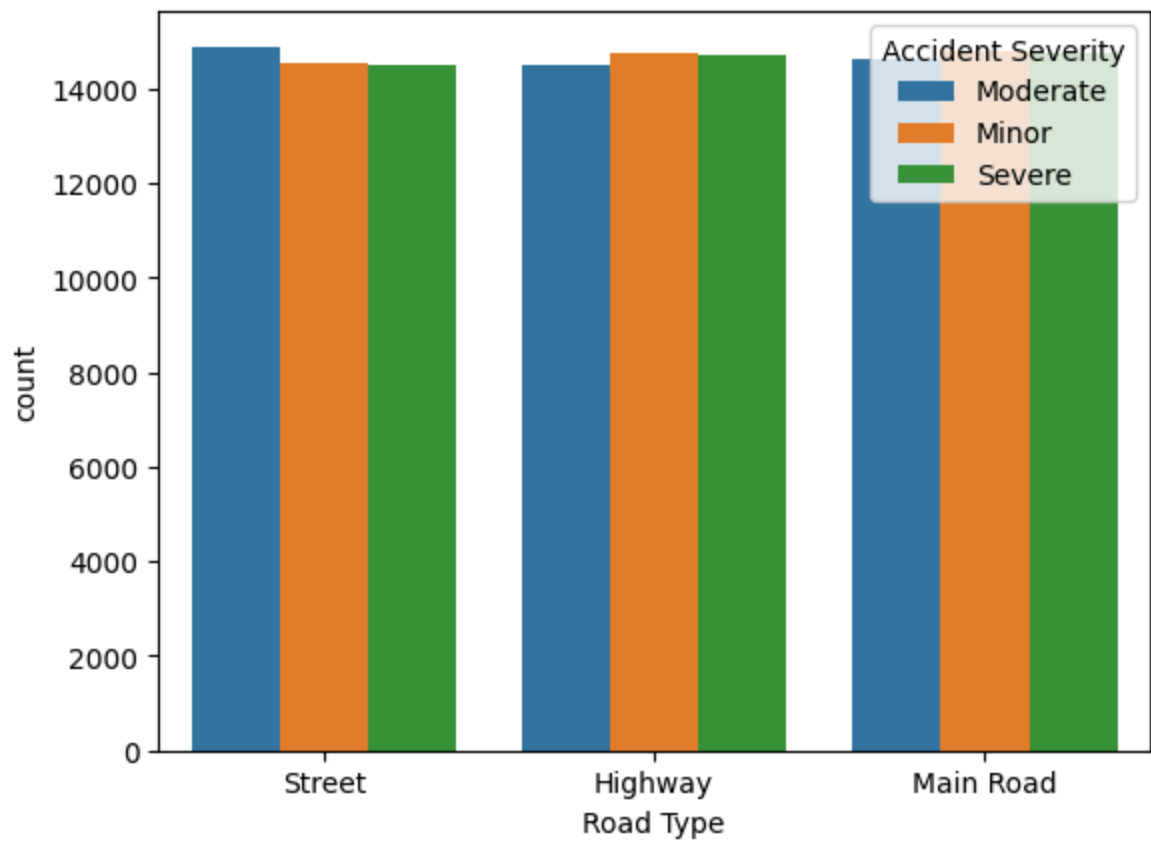
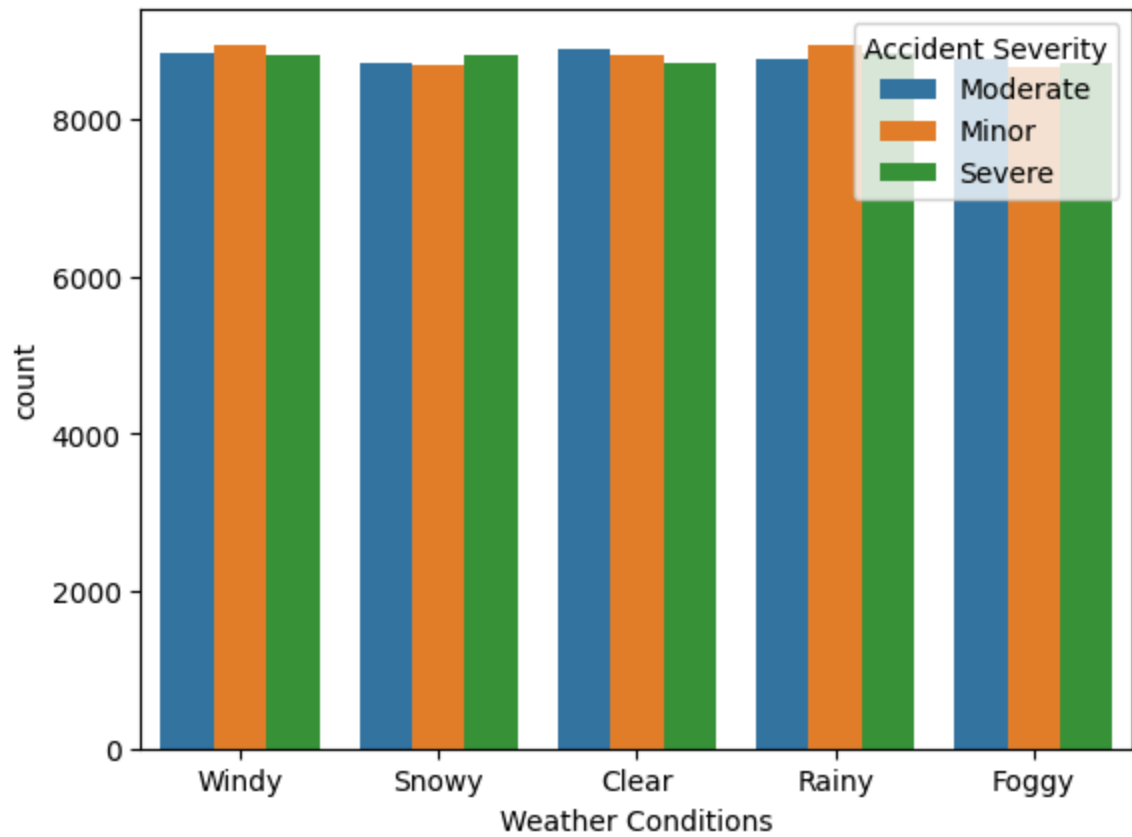
```
In [56]: df['Driver Alcohol Level'] = df['Driver Alcohol Level'].fillna(0)
df['Emergency Response Time'] = df['Emergency Response Time'].fillna(df['Emergency Response Time'].median())
df['Medical Cost'] = df['Medical Cost'].fillna(df['Medical Cost'].median())
```

Question 3d Driver Alcohol Level may be MAR because alcohol tests might not be conducted in some regions or hospitals due to operational reasons. In this case, the missing value depends on external factors such as location or reporting practices.

Driver Alcohol Level may also be MNAR because drivers with higher alcohol consumption may refuse testing or leave the accident scene. Here, the missingness is directly related to the alcohol level itself.

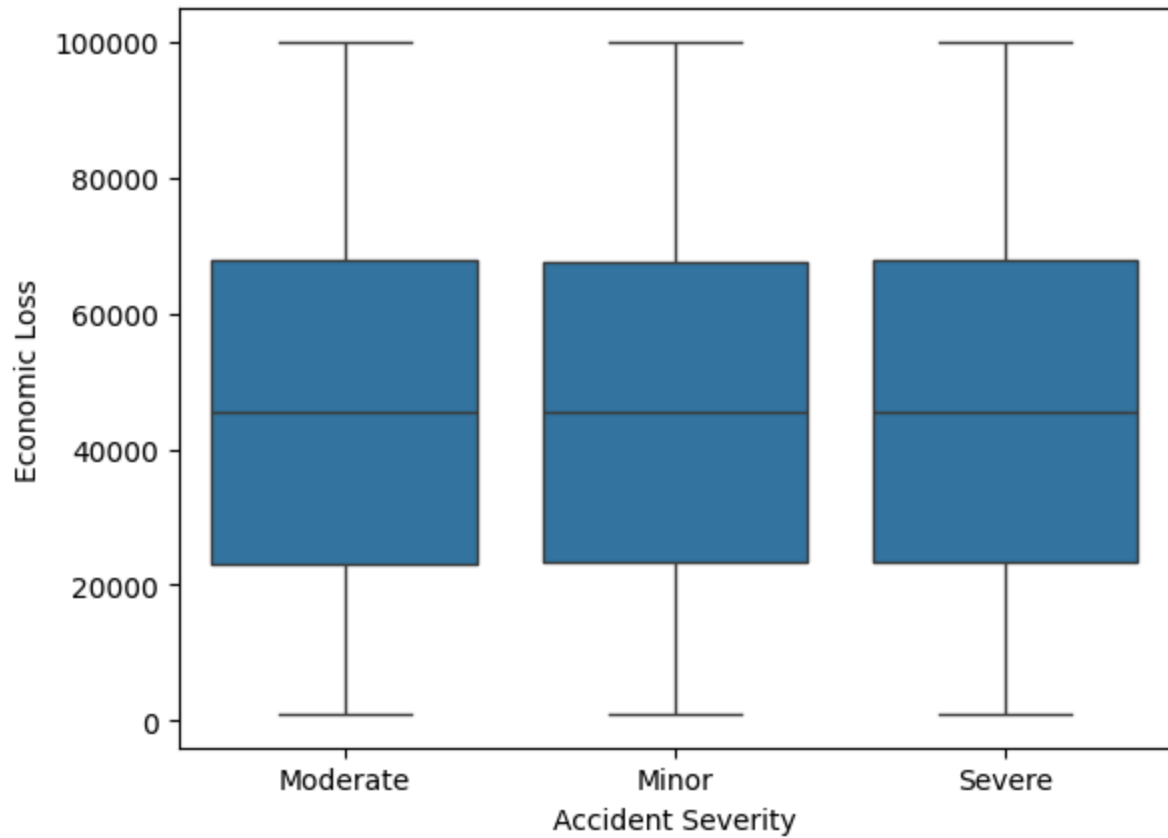
question 4a

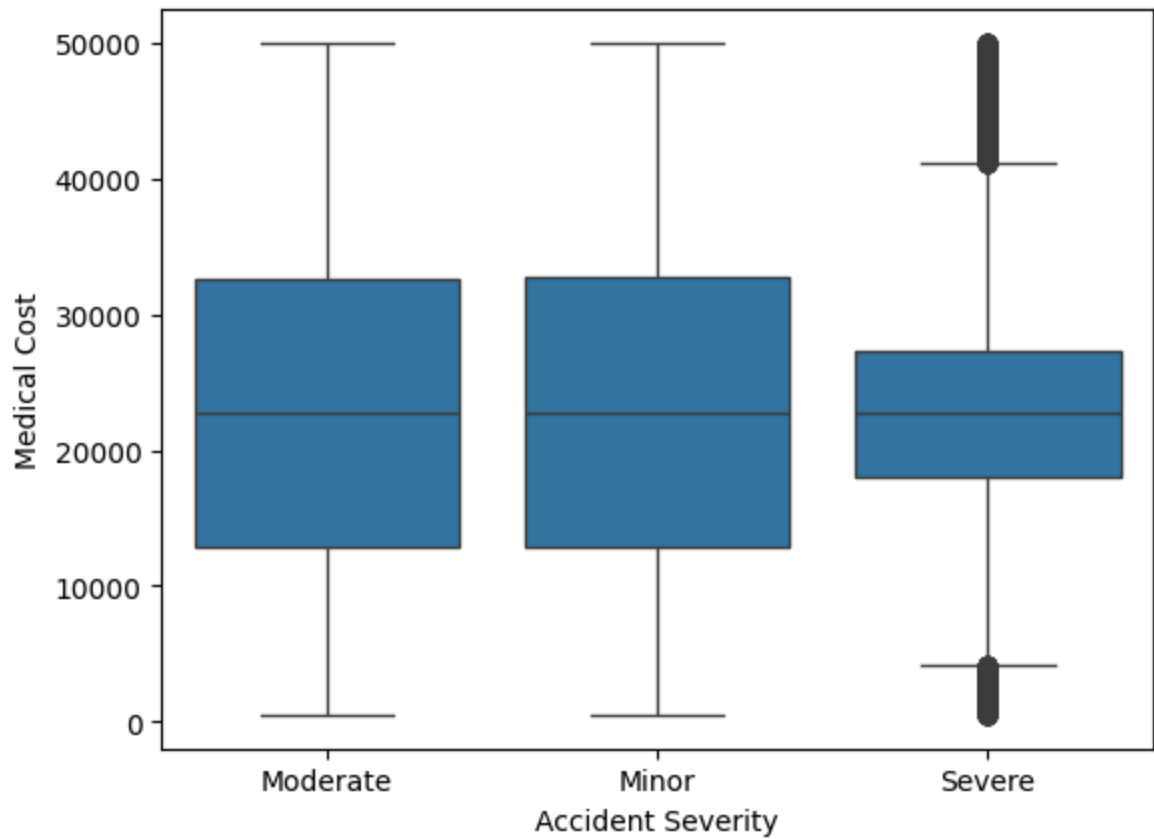
```
In [57]: import seaborn as sns
import matplotlib.pyplot as plt
sns.countplot(data=df, x="Weather Conditions", hue="Accident Severity")
plt.show()
sns.countplot(data=df, x="Road Type", hue="Accident Severity")
plt.show()
```



Question 4b

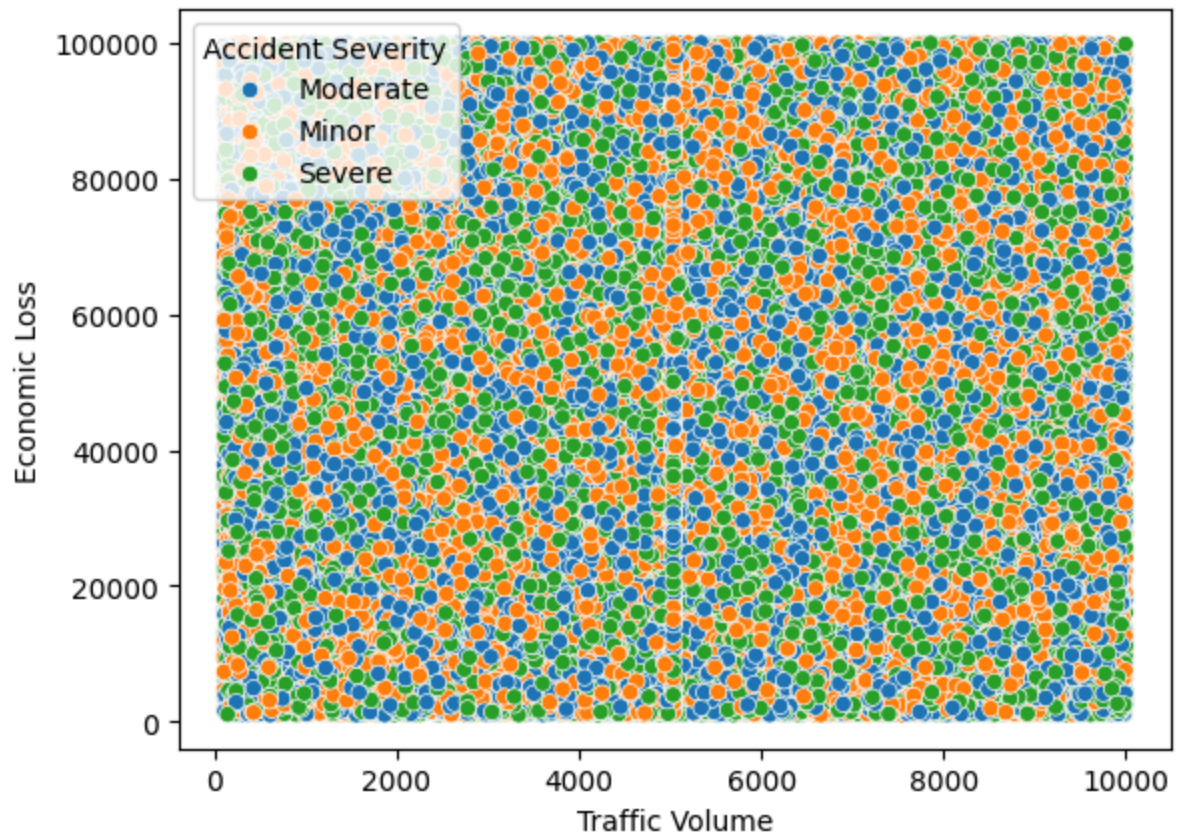

```
In [58]: sns.boxplot(data=df, x="Accident Severity", y="Economic Loss")  
plt.show()  
sns.boxplot(data=df, x="Accident Severity", y="Medical Cost")  
plt.show()
```





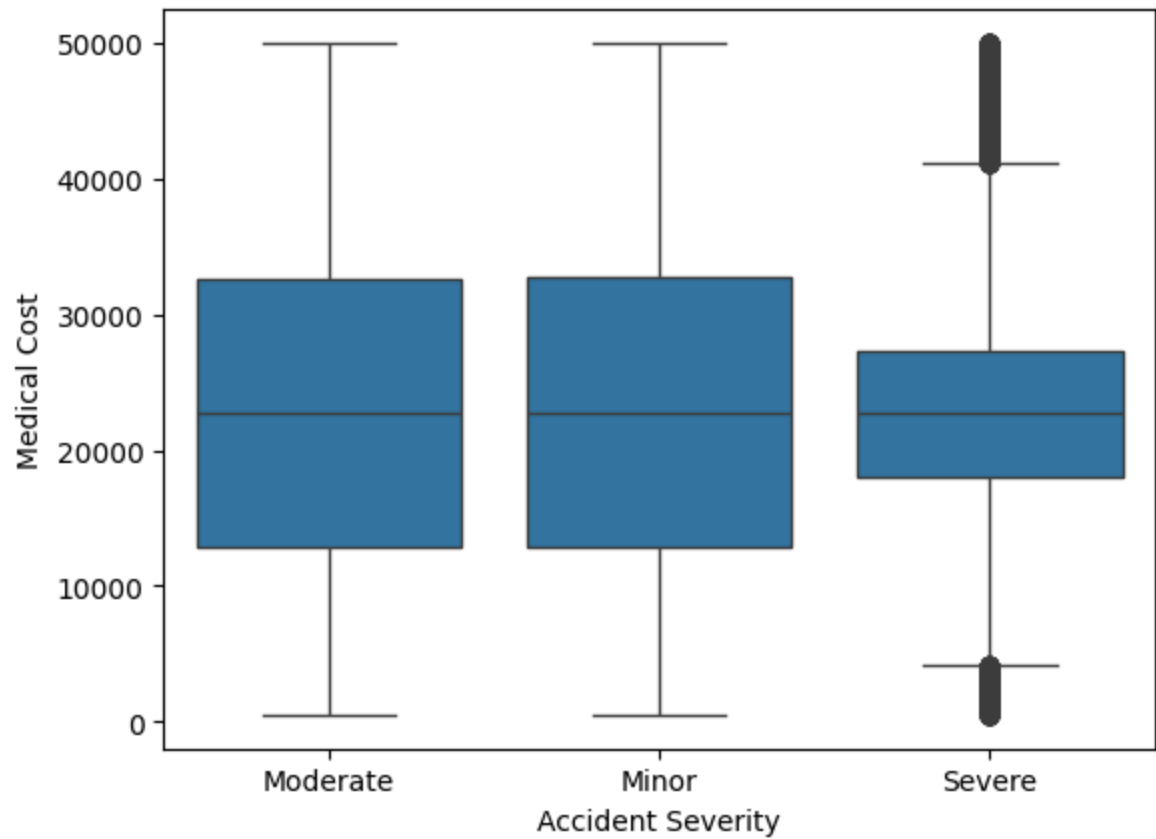
Question 4c

```
In [59]: sns.scatterplot(data=df, x="Traffic Volume", y="Economic Loss", hue="Accident Severity",  
plt.show())
```



Question 4d (Box Plot)

```
In [60]: sns.boxplot(data=df, x="Accident Severity", y="Medical Cost")  
plt.show()
```



Question 5

```
In [61]: corr = df[num_vars].corr()
print(corr)

plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

	Year	Visibility Level \
Year	1.000000	0.002552
Visibility Level	0.002552	1.000000
Number of Vehicles Involved	-0.003528	0.000816
Speed Limit	-0.002744	-0.002049
Driver Alcohol Level	0.000554	0.004305
Driver Fatigue	0.000545	0.002185
Pedestrians Involved	-0.002093	-0.004019
Cyclists Involved	-0.000077	-0.000089
Number of Injuries	0.001120	0.000434
Number of Fatalities	-0.004965	-0.001022
Emergency Response Time	-0.001566	-0.000918
Traffic Volume	-0.000343	-0.003100
Insurance Claims	0.001109	0.004462
Medical Cost	-0.006863	0.002647
Economic Loss	-0.004264	0.003311
Population Density	0.003751	-0.000308

	Number of Vehicles Involved	Speed Limit \
Year	-0.003528	-0.002744
Visibility Level	0.000816	-0.002049
Number of Vehicles Involved	1.000000	0.003759
Speed Limit	0.003759	1.000000
Driver Alcohol Level	0.004617	-0.002032
Driver Fatigue	0.001789	-0.001007
Pedestrians Involved	-0.004993	-0.001365
Cyclists Involved	-0.001884	0.001922
Number of Injuries	0.002234	-0.001118
Number of Fatalities	0.003353	0.000621
Emergency Response Time	0.002753	-0.000310
Traffic Volume	0.003952	-0.000994
Insurance Claims	-0.003621	0.000065
Medical Cost	-0.002471	0.000777
Economic Loss	-0.001200	0.000733
Population Density	-0.004531	0.001577

	Driver Alcohol Level	Driver Fatigue \
Year	0.000554	0.000545
Visibility Level	0.004305	0.002185
Number of Vehicles Involved	0.004617	0.001789
Speed Limit	-0.002032	-0.001007
Driver Alcohol Level	1.000000	-0.004203
Driver Fatigue	-0.004203	1.000000
Pedestrians Involved	0.001666	-0.001152
Cyclists Involved	-0.000918	-0.001007
Number of Injuries	-0.004300	-0.004839
Number of Fatalities	0.001312	0.002685
Emergency Response Time	0.001977	0.001180
Traffic Volume	0.005964	-0.003659
Insurance Claims	-0.006502	0.002343
Medical Cost	-0.000729	-0.000363
Economic Loss	-0.001484	0.005159
Population Density	0.002430	0.003296

	Pedestrians Involved	Cyclists Involved \
Year	-0.002093	-0.000077

Visibility Level	-0.004019	-0.000089
Number of Vehicles Involved	-0.004993	-0.001884
Speed Limit	-0.001365	0.001922
Driver Alcohol Level	0.001666	-0.000918
Driver Fatigue	-0.001152	-0.001007
Pedestrians Involved	1.000000	0.002873
Cyclists Involved	0.002873	1.000000
Number of Injuries	-0.008377	0.000406
Number of Fatalities	0.003346	0.005160
Emergency Response Time	0.004674	-0.000784
Traffic Volume	0.001030	-0.001688
Insurance Claims	0.001225	0.000451
Medical Cost	-0.000438	-0.004976
Economic Loss	0.000481	-0.002048
Population Density	0.000845	0.004140

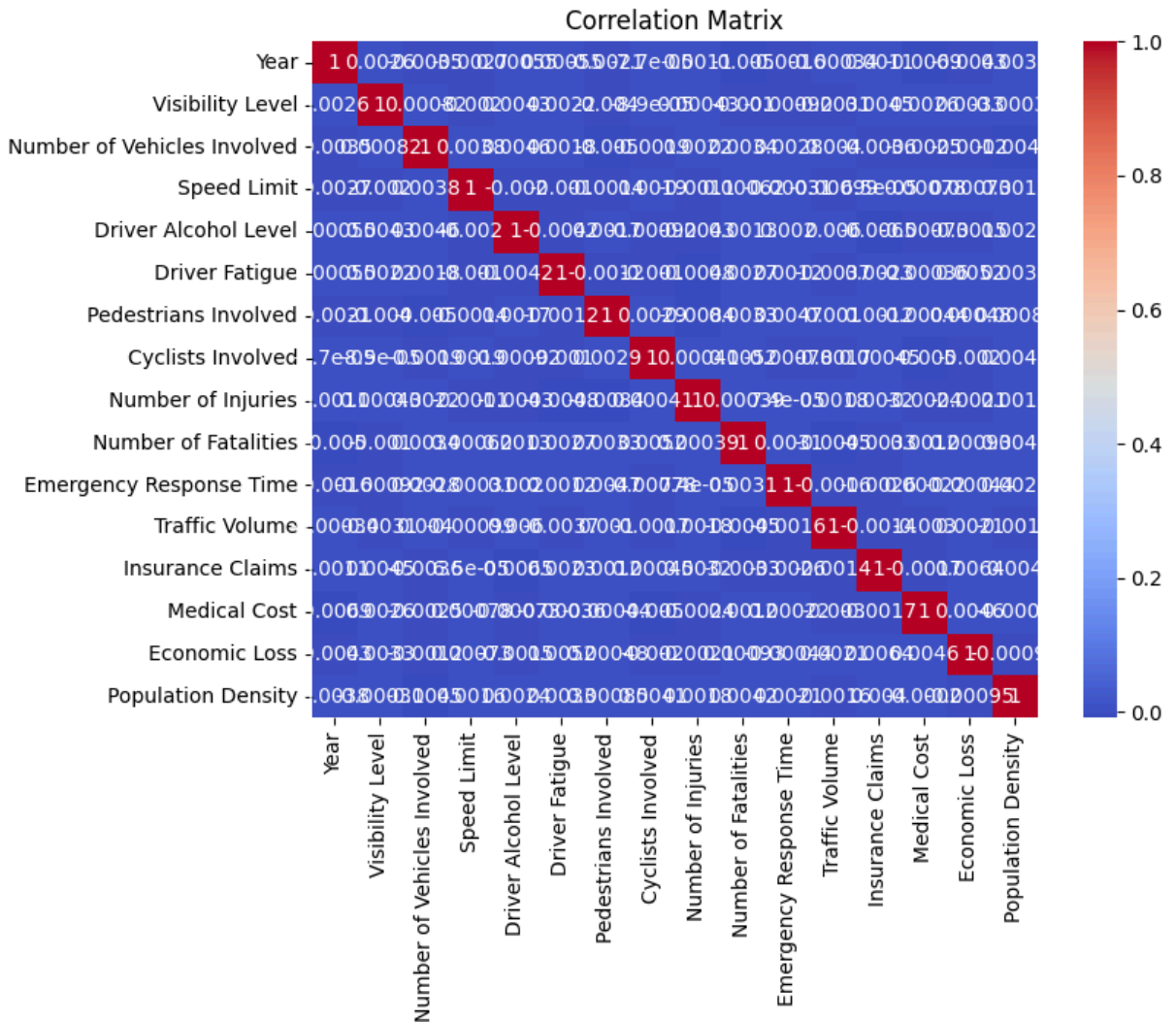
	Number of Injuries	Number of Fatalities \
Year	0.001120	-0.004965
Visibility Level	0.000434	-0.001022
Number of Vehicles Involved	0.002234	0.003353
Speed Limit	-0.001118	0.000621
Driver Alcohol Level	-0.004300	0.001312
Driver Fatigue	-0.004839	0.002685
Pedestrians Involved	-0.008377	0.003346
Cyclists Involved	0.000406	0.005160
Number of Injuries	1.000000	0.000389
Number of Fatalities	0.000389	1.000000
Emergency Response Time	0.000074	0.003098
Traffic Volume	0.001753	-0.004459
Insurance Claims	0.003242	-0.003330
Medical Cost	-0.002365	0.001212
Economic Loss	-0.002092	0.000927
Population Density	0.001812	0.004247

	Emergency Response Time	Traffic Volume \
Year	-0.001566	-0.000343
Visibility Level	-0.000918	-0.003100
Number of Vehicles Involved	0.002753	0.003952
Speed Limit	-0.000310	-0.000994
Driver Alcohol Level	0.001977	0.005964
Driver Fatigue	0.001180	-0.003659
Pedestrians Involved	0.004674	0.001030
Cyclists Involved	-0.000784	-0.001688
Number of Injuries	0.000074	0.001753
Number of Fatalities	0.003098	-0.004459
Emergency Response Time	1.000000	-0.001588
Traffic Volume	-0.001588	1.000000
Insurance Claims	-0.002579	-0.001430
Medical Cost	0.000221	-0.002987
Economic Loss	-0.000436	0.002088
Population Density	0.002107	-0.001591

	Insurance Claims	Medical Cost	Economic Loss \
Year	0.001109	-0.006863	-0.004264
Visibility Level	0.004462	0.002647	0.003311
Number of Vehicles Involved	-0.003621	-0.002471	-0.001200

Speed Limit	0.000065	0.000777	0.000733
Driver Alcohol Level	-0.006502	-0.000729	-0.001484
Driver Fatigue	0.002343	-0.000363	0.005159
Pedestrians Involved	0.001225	-0.000438	0.000481
Cyclists Involved	0.000451	-0.004976	-0.002048
Number of Injuries	0.003242	-0.002365	-0.002092
Number of Fatalities	-0.003330	0.001212	0.000927
Emergency Response Time	-0.002579	0.000221	-0.000436
Traffic Volume	-0.001430	-0.002987	0.002088
Insurance Claims	1.000000	-0.001716	0.006437
Medical Cost	-0.001716	1.000000	0.004607
Economic Loss	0.006437	0.004607	1.000000
Population Density	0.003998	-0.000205	-0.000952

	Population Density
Year	0.003751
Visibility Level	-0.000308
Number of Vehicles Involved	-0.004531
Speed Limit	0.001577
Driver Alcohol Level	0.002430
Driver Fatigue	0.003296
Pedestrians Involved	0.000845
Cyclists Involved	0.004140
Number of Injuries	0.001812
Number of Fatalities	0.004247
Emergency Response Time	0.002107
Traffic Volume	-0.001591
Insurance Claims	0.003998
Medical Cost	-0.000205
Economic Loss	-0.000952
Population Density	1.000000



Question 6a

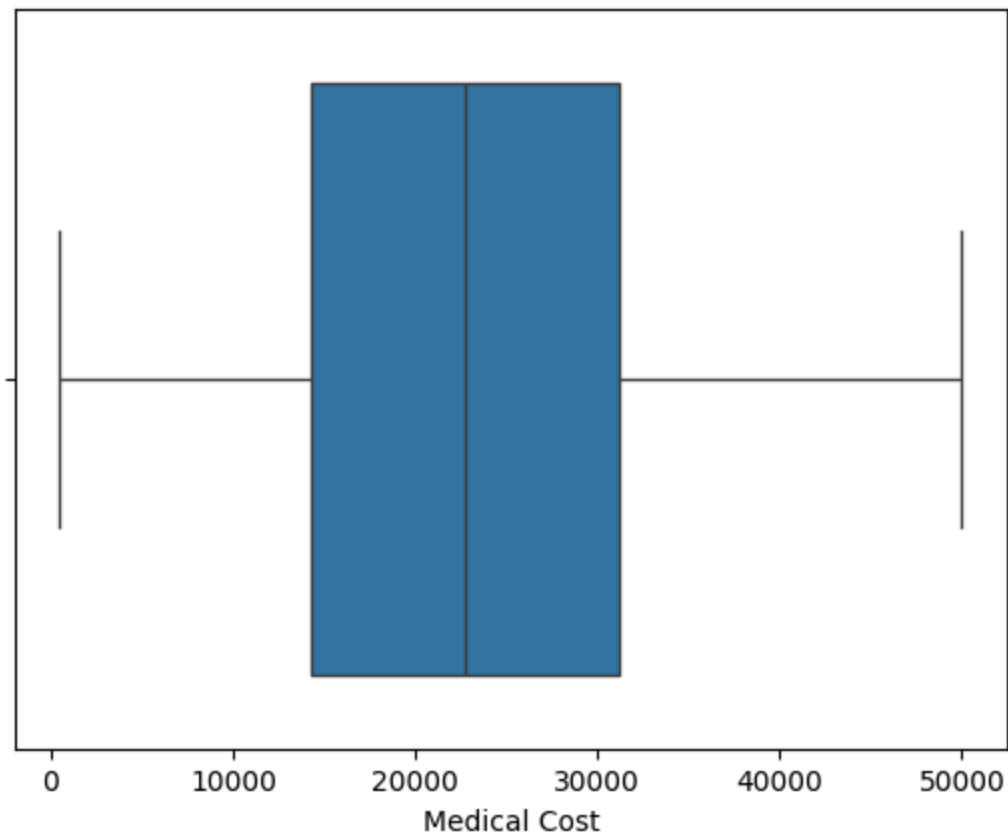
```
In [ ]: # finding Outlier using IQR method
for col in ['Medical Cost', 'Economic Loss', 'Traffic Volume', 'Emergency Response Time']:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    print(f"{col}: Outliers count = {len(outliers)}")
```

Medical Cost: Outliers count = 0
 Economic Loss: Outliers count = 0
 Traffic Volume: Outliers count = 0
 Emergency Response Time: Outliers count = 0

Question 6b

```
In [63]: df['Medical Cost'].sort_values(ascending=False).head(10)
sns.boxplot(x=df['Medical Cost'])
plt.show()
```

Question 7a

```
In [ ]: # Creating a Date column from Year + Month
df['Date'] = pd.to_datetime(df['Year'].astype(str) + " " + df['Month'], errors='coerce')

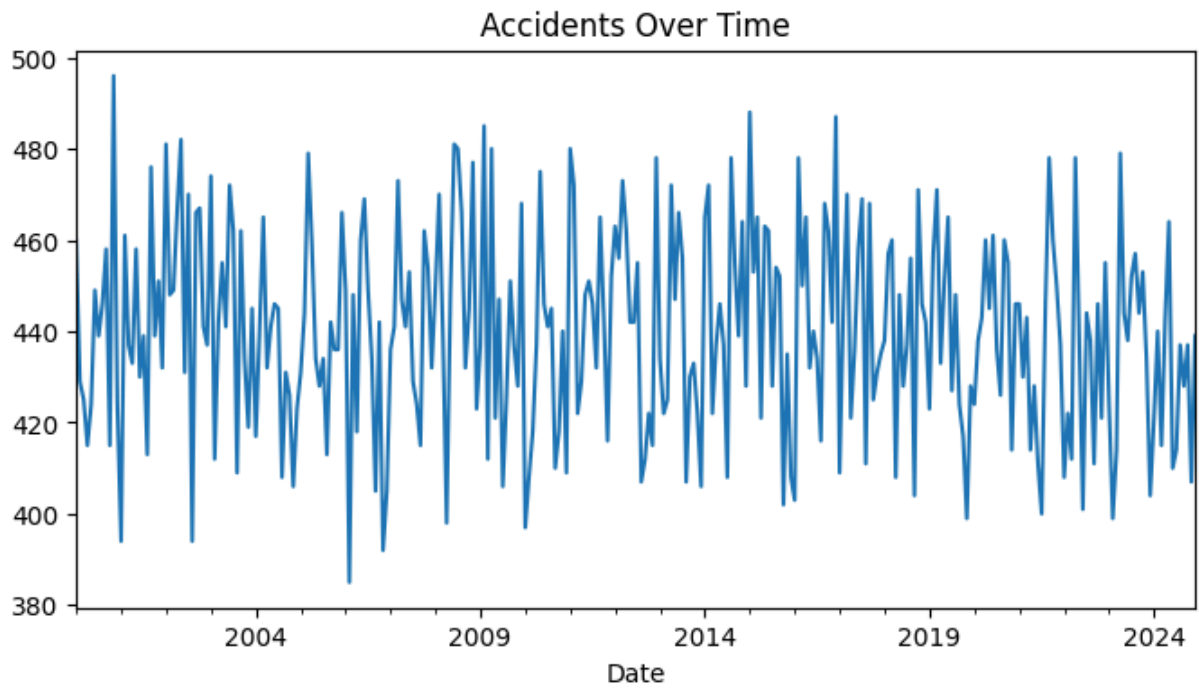
# To count accidents per month
accidents_over_time = df.groupby(df['Date'].dt.to_period("M")).size()

# Plot
accidents_over_time.plot(kind='line', figsize=(8,4), title="Accidents Over Time")
```

C:\Users\dell\AppData\Local\Temp\ipykernel_56908\4128321797.py:2: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

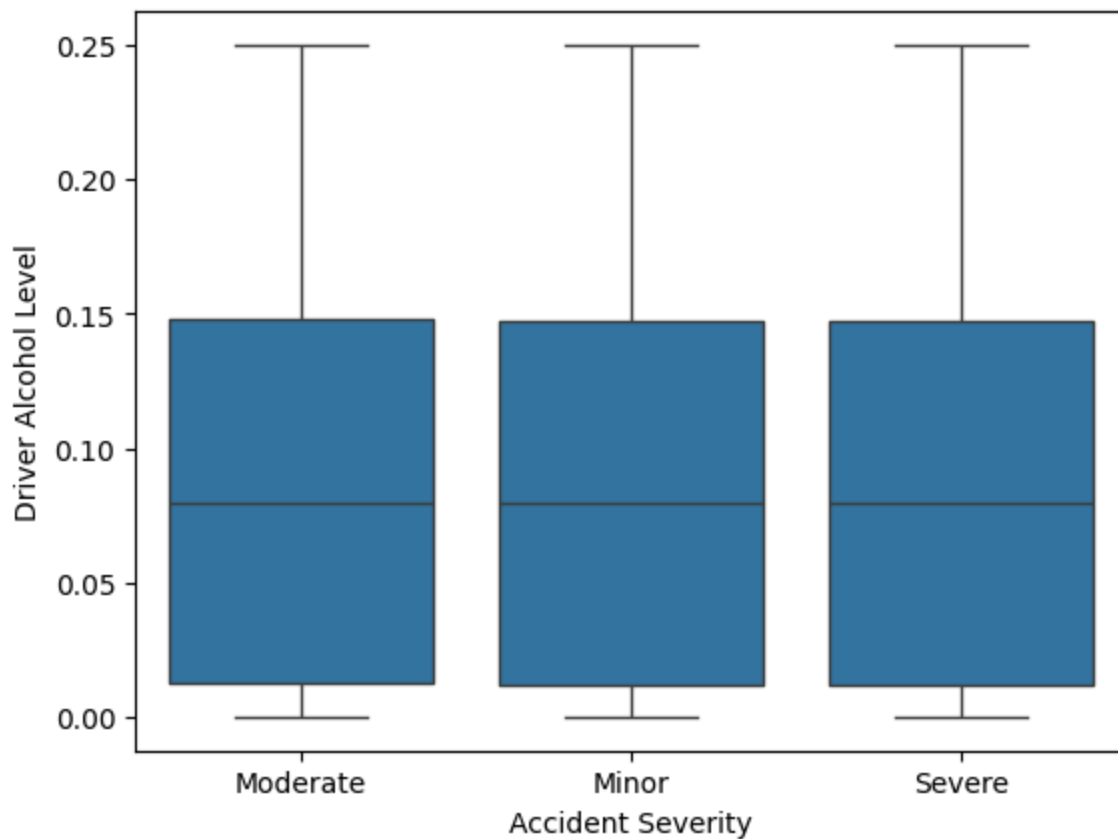
```
df['Date'] = pd.to_datetime(df['Year'].astype(str) + " " + df['Month'], errors='coerce')
```

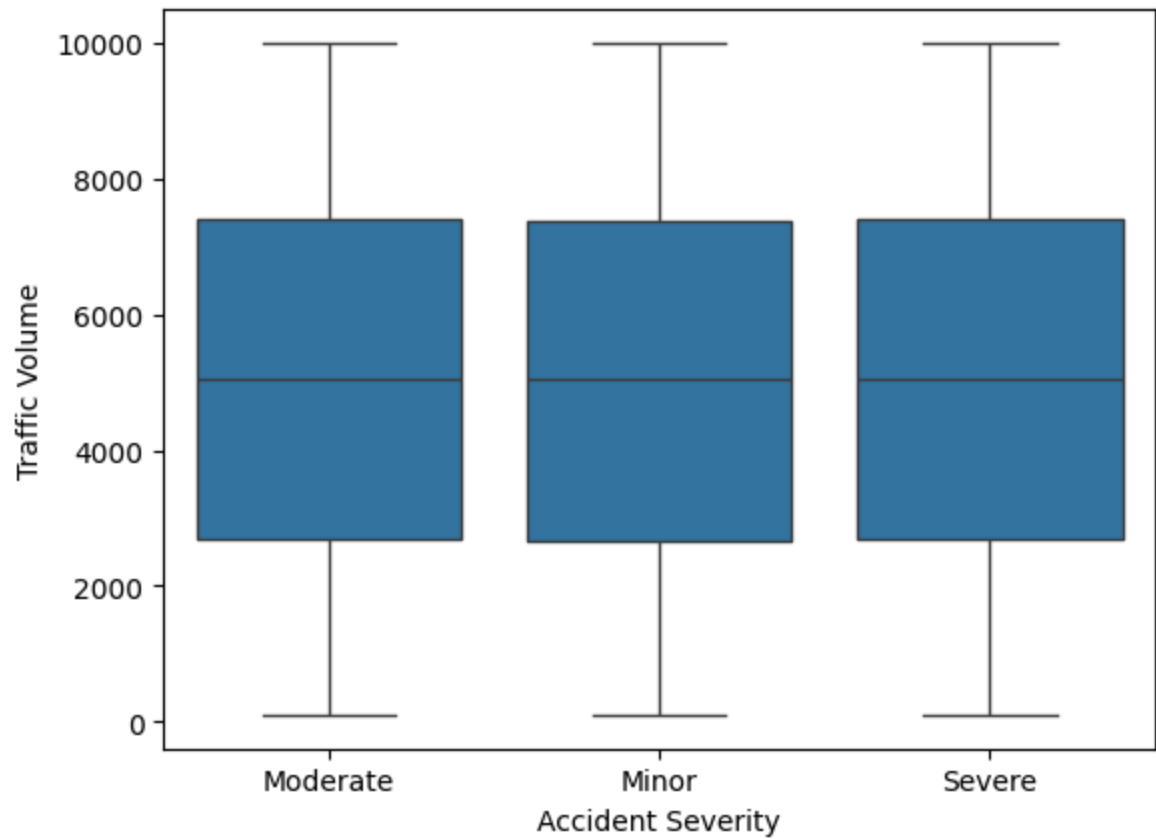
```
Out[ ]: <Axes: title={'center': 'Accidents Over Time'}, xlabel='Date'>
```



Question 7b

```
In [65]: sns.boxplot(data=df, x="Accident Severity", y="Driver Alcohol Level")  
plt.show()  
sns.boxplot(data=df, x="Accident Severity", y="Traffic Volume")  
plt.show()
```

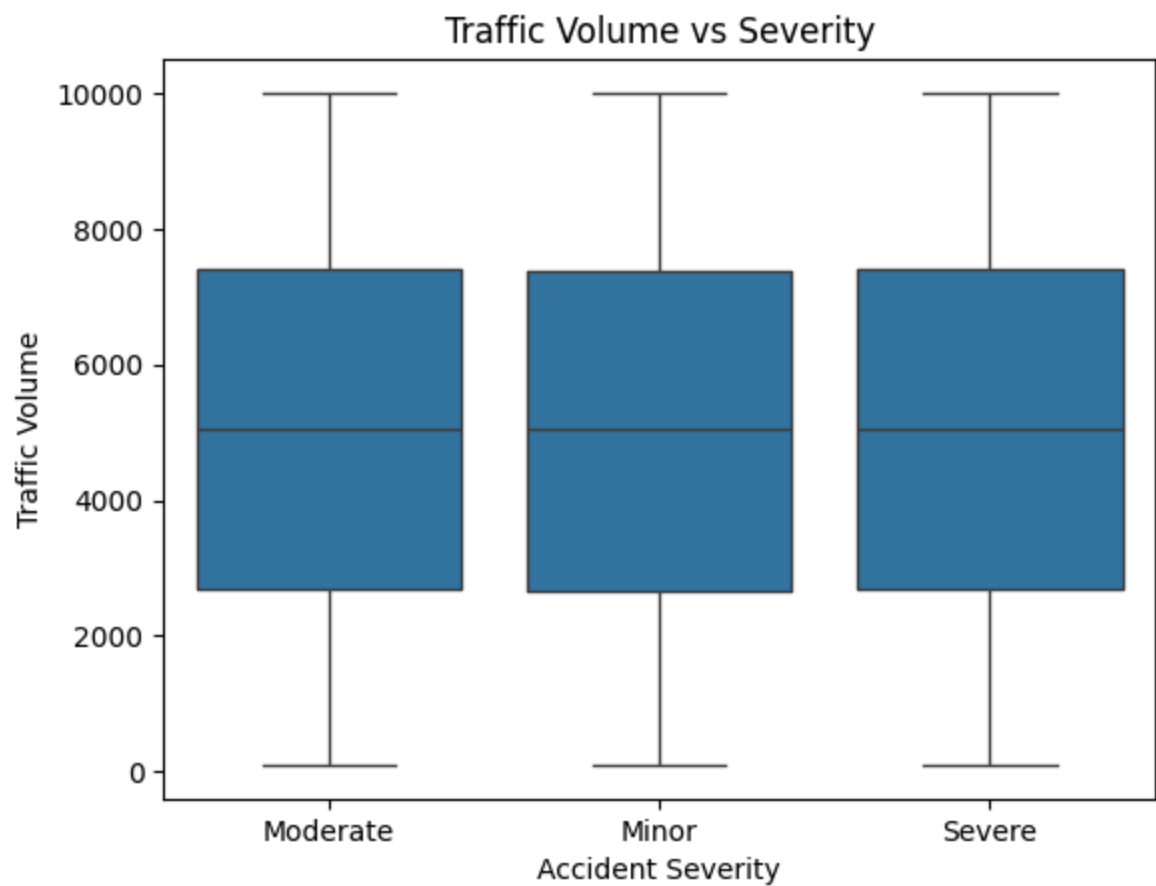
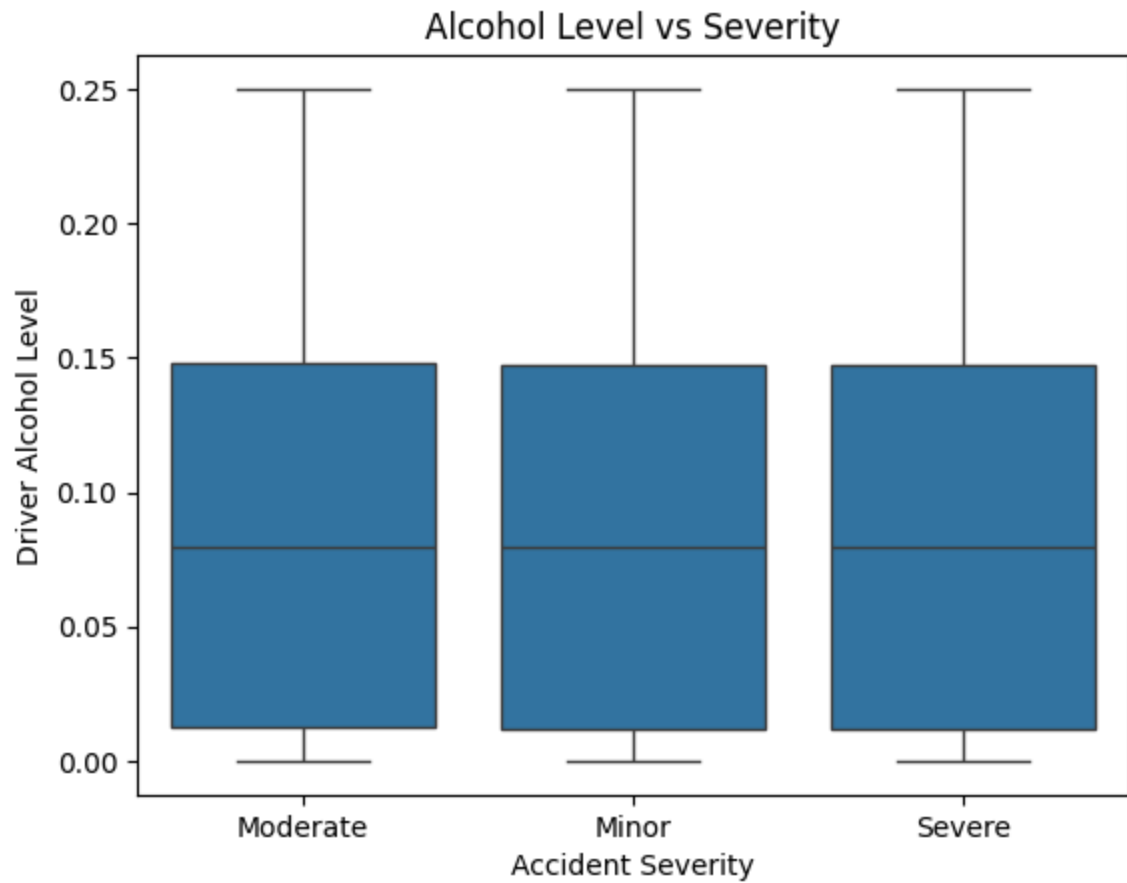




Question 7c

```
In [66]: sns.boxplot(data=df, x="Accident Severity", y="Driver Alcohol Level")
plt.title("Alcohol Level vs Severity")
plt.show()

sns.boxplot(data=df, x="Accident Severity", y="Traffic Volume")
plt.title("Traffic Volume vs Severity")
plt.show()
```



Question 7d

```
In [67]: sns.countplot(data=df, x="Region", hue="Accident Severity")  
plt.show()  
sns.boxplot(data=df, x="Region", y="Economic Loss")  
plt.show()
```

